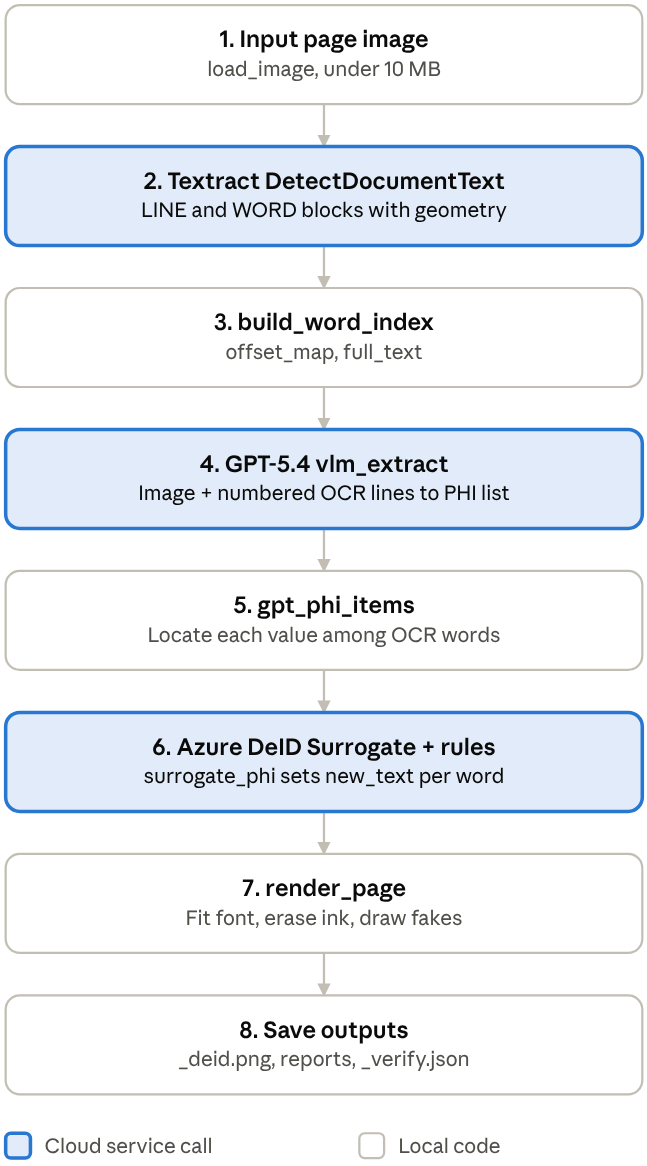
v9 Pipeline Structure

Oct 30, 2026

The v9 notebook de-identifies a scanned page in eight stages: Textract finds every word and its position, GPT-5.4 lists the PHI, Azure DeID writes the fakes, and local code draws them back onto the image.

At a glance

Three cloud calls, everything else runs locally



v9 pipeline · 8 stages, 3 cloud calls

Only stages 2, 4 and 6 leave the machine; positions come from Textract, the PHI decision from GPT, and the fake values from Azure DeID plus local rules.

Data handed between stages

Every stage keys on Textract's `word_id`, so positions found in stage 2 travel unchanged to the drawing in stage 7.

#	Stage	Notebook section	Runs on	Input	Output	Key functions
1	Input	3	Local	Image path	Validated path, image bytes	`load_image`
2	OCR	4	AWS Textract	Image bytes (< 10 MB)	`Blocks`: PAGE, LINE, WORD with geometry, confidence, `TextType`	`get_ocr`
3	Word index	6	Local	`Blocks`, image width and height	`full_text`, `offset_map`, `word_by_id`	`build_word_index`
4	PHI list	11b	Azure OpenAI GPT-5.4	Image (≤ 2,000 px wide) + numbered OCR lines	`gpt_answer`: `fields[]`, `free_text_phi[]`	`ocr_text_for_gpt`, `vlm_extract`, `_parse_json`
5	Locate	11b	Local	`gpt_answer`, `offset_map`	`phi_items` (words, spans, kind), `unmatched_phi`	`gpt_phi_items`, `match_text`
6	Fakes	11, 12	Azure DeID + local rules	`phi_items`	`new_text` per word, `flags`, `entities`, `phi_stats`	`build_phi_text`, `deid_surrogate_text`, `surrogate_phi`
7	Render	13	Local	Image, `offset_map`, `flags`	De-identified image, `actions` per word	`render_page`, `choose_styles`, `erase_word`, `draw_word`
8	Outputs	13b, 14	Local (+ Textract for verification)	Everything above	`_deid.png`, `_comparison.png`, text files, `_deid_report.xlsx`, `_verify.json`, batch files	`save_text_files`, `save_excel_report`, `save_verification_record`

Stages in detail

1–2. Input and OCR

- `load_image` checks the path exists and the extension is in `SUPPORTED_IMAGE_FORMATS`.
- `get_ocr(image_bytes)` calls Textract `detect_document_text` (synchronous API, under 10 MB) through the company `aws_client`; `@aws_client.refresh_on_expire` renews credentials.
- Textract returns `Blocks` of type `PAGE`, `LINE` and `WORD`, with normalised geometry, confidence and `TextType` (printed or handwriting). It returns no form key–value pairs; GPT supplies that context.

3. Word index

`build_word_index(blocks, W, H)` walks every LINE and its child WORDs and returns:

- `full_text`: the page as line-ordered plain text;
- `offset_map`: one dict per word with `word_id`, `text`, `start`/`end` offsets in `full_text`, `line_idx`, pixel box `px`, `poly`, skew `angle`, `confidence` and `handwriting`.

This is the single source of truth for positions.

4. PHI list (GPT-5.4)

- `ocr_text_for_gpt` serialises `offset_map` as `L<n>: …` lines.
- `vlm_extract` makes one chat-completions call with the downscaled image and those lines, then `_parse_json` reads the strict JSON reply.
- `fields[]`: label, value, category, `is_phi`, OCR line. `free_text_phi[]`: identifiers outside labelled boxes.
- GPT only classifies what is PHI; it never decides positions.

5. Locate

`gpt_phi_items` maps each PHI value onto OCR words with `match_text`:

- an n±1-word sliding window on one OCR line;
- `_compact` normalisation (lowercase, letters and digits only);
- `SequenceMatcher` similarity ⩾ 0.85, exact match for values of 3 characters or fewer.

It prefers GPT's line, then the copy nearest the label; a `used` set keeps each word in one value. Output: `phi_items` (label, kind, `words`, `spans` for glued tokens like `DOB:03/15/1980`) and `unmatched_phi` (values not located, which stay unchanged on the image).

6. Fakes (Azure DeID + rules)

1. `build_phi_text` writes one `LABEL: value` line per item.
2. `deid_surrogate_text` makes one Azure DeID `Surrogate` call; if the answer's line count differs, each value is resent on its own.
3. `surrogate_phi`, per item:
   - reuses an earlier fake from `SURROGATE_MEMO`;

    - aligns fake tokens to the OCR words with `_pieces_from_fake` (`SequenceMatcher` opcodes);
    - applies rule overrides: `shift_date` for missing or invalid date fakes, `realistic_email` for e-mails, then `MISSED_PHI_ACTION` (keep or erase) for values Azure left unchanged;
    - shapes case and punctuation with `_shape_like`;
    - propagates the fake to exact repeats on the page.
4. Result: `word["new_text"]` on every changed word, plus `flags`, `entities` and `phi_stats`.

7. Render

- `choose_styles` fits a font and size per line by rendering the **original** text and scoring Dice overlap against the Otsu ink mask.
- `erase_word` replaces the ink, but not form lines, with background pixels sampled nearby.
- `draw_word` blends `new_text` at the original baseline in an ink colour matched to the original's darkness.
- Run layout handles length mismatch: grow into free space → carry neighbouring words → trim words Azure added → shrink the font → condense.

8. Outputs

- `_deid.png` (de-identified page) and `_comparison.png` (original next to output).
- `_phi_text_sent.txt`, `_phi_text_deid.txt`, `_changes.txt`, `_surrogated.txt`, `_gpt.json` and `_deid_report.xlsx`.
- `_verify.json`: the output re-read by Textract, for the `phi_leak_verification` notebook.
- Batch: `batch_summary.csv`, `batch_errors.csv`, `batch_changes.xlsx`.

Orchestration and service calls

The single-image cells and the batch run the same eight stages; in production a page costs one Textract, one GPT and one Azure DeID call.

Two ways the stages run

- **Single image (sections 4–13b):** each cell runs one stage on global variables (`offset_map`, `phi_items`, `flags` …) for `IMAGE_PATH`, with debug printouts.
- **Batch (section 14):** `process_image_pipeline` repeats the same sequence per page with local variables. The batch loop wraps it in `try/except` per page, clears `SURROGATE_MEMO` once per batch so a value keeps one fake across pages, and aggregates the summary rows.

Service calls per page

Service	Calls	When there are more
AWS Textract	1	+1 when `SAVE_VERIFICATION_RECORD` re-reads the output
Azure OpenAI GPT-5.4	1	Never (re-runs call again unless `VLM_CACHE_DIR` is set)
Azure DeID	1	One per value when Azure's line count does not match; +1 with `CHECK_PAGE_TEXT`